Informe SP4 y SP5: Integración con JavaScript

Proyecto: Sísifo Gym (sitio web front-end de reserva de clases y venta de pases)
Materia: Aplicaciones Web 1 - Sistemas - Prof. Andrés Senn (2026s2)
Alumno: David Gabriel Jerez

1. Resumen de lo desarrollado

Se incorporó JavaScript al sitio estático del SP3 para que las páginas Clases, Pases, Mi Cuenta y Contacto obtengan sus datos de archivos JSON con **fetch**, los dibujen en pantalla y respondan a las acciones del usuario. Se completó el ciclo de reserva de clases (selección, confirmación y cancelación) y el de compra de un pase, ambos con pantallas de confirmación. El código se dividió en módulos ES (import/export) según su responsabilidad y el CSS se amplió con los componentes nuevos, manteniendo la nomenclatura BEM del SP3. También se resolvieron los dos pendientes que el informe del SP3 dejaba para esta etapa: el menú desplegable en móvil y el paso de los datos del HTML a un JSON.

Punto	Qué se hizo	Dónde
4.1 Eventos	Filtros (submit, change, reset), reserva, compra y cancelación (click), formulario de contacto (submit, blur, input) y menú móvil (click, Escape).	`js/paginas/*.js`, `js/componentes/menu.js`
4.2 JSON	Tres archivos con los datos a renderizar y un objeto JSON para la consulta de contacto.	`data/*.json`
4.3 fetch	Una única función que descarga y valida cada archivo, con manejo de errores y caché.	`js/modulos/api.js`
4.4 Renderizado	Tarjetas de clases, tabla de pases, resumen de reservas, datos de usuario y reservas de Mi Cuenta se dibujan desde el JSON.	`js/paginas/*.js`
4.5 Validaciones	Validación del formulario de contacto con mensajes bajo cada campo.	`js/modulos/validacion.js`, `js/paginas/contacto.js`
4.6 Directorios	Recursos separados en css, js (con tres subcarpetas), data y assets.	Sección 3
4.7 Modularización	16 archivos JavaScript: módulos ES con una responsabilidad cada uno y carga dinámica por página.	Sección 9
4.8 Informe	Este documento, con los cambios respecto de la entrega anterior.	Sección 11

2. Cómo ejecutar el proyecto

El sitio debe abrirse desde un servidor HTTP y no con doble clic sobre *index.html*: los navegadores bloquean fetch y los módulos ES cuando la página se carga con el protocolo file://. Alternativas:

- Extensión **Live Server** de Visual Studio Code (botón "Go Live").
- Desde la carpeta del proyecto: `python -m http.server 8000` y abrir `http://localhost:8000`.
- Desde la carpeta del proyecto: `npx serve`.

Las reservas, la selección en curso y el plan activo se guardan en *localStorage* (clave `sisifo-gym:estado:v1`). Para volver al estado inicial alcanza con borrar los datos del sitio o ejecutar

`localStorage.clear()` en la consola.

3. Organización de directorios

```
sisifo-gym/
|-- index.html, clases.html, pases.html,
|   contacto.html, mi-cuenta.html     (en la raiz, como exige el SP2)
|-- assets/img/                       (logo y fotos de clases)
|-- css/
|   `-- styles.css                    (unica hoja de estilos, ampliada)
|-- data/                             (datos que simulan la API REST)
|   |-- clases.json
|   |-- pases.json
|   `-- usuario.json
`-- js/
    |-- main.js                       (punto de entrada de todas las paginas)
    |-- modulos/                      (logica sin interfaz)
    |   |-- api.js          almacenamiento.js   dom.js
    |   |-- estado.js       formato.js          reglas.js     validacion.js
    |-- componentes/                  (piezas de interfaz reutilizables)
    |   |-- avisos.js       dialogo.js          menu.js       mensaje-estado.js
    `-- paginas/                      (un controlador por pagina)
        |-- clases.js       pases.js            cuenta.js     contacto.js
```

Las dependencias van en una sola dirección: las páginas usan componentes y módulos, y los módulos no conocen a las páginas. Los módulos `reglas.js`, `formato.js` y `validacion.js` son funciones puras (no tocan el DOM, el almacenamiento ni la red), por lo que se pueden probar de forma aislada.

4. Datos en JSON (punto 4.2)

Los datos que en el SP3 estaban escritos en el HTML pasaron a tres archivos dentro de *data/*, que cumplen el papel de una API REST:

Archivo	Contenido	Equivale a
`clases.json`	Disciplinas y clases: nombre, días, horario, profesor, cupos totales, cupos ocupados por día e imagen.	`GET /clases`
`pases.json`	Planes con nombre, acceso, precio y moneda.	`GET /pases`
`usuario.json`	Usuario inicial: nombre, plan activo y reservas existentes. Es la "semilla" del estado.	`GET /usuario`

```
{
  "id": "crossfit-0800",
  "nombre": "CrossFit",
  "disciplina": "crossfit",
  "dias": ["lunes", "miercoles"],
  "horario": "08:00",
  "profesor": { "rol": "Profesor", "nombre": "Marcos Pérez" },
  "cuposTotales": 15,
  "cuposOcupados": { "lunes": 10, "miercoles": 8 },
  "imagen": { "src": "assets/img/crossfit.jpg", "alt": "CrossFit" }
}
```

Cálculo de cupos. Cupos disponibles = cupos totales - cupos ocupados ese día - reservas confirmadas del usuario en esa sesión. Con los datos cargados se reproducen exactamente los valores del diseño original: "4 de 15" (CrossFit, lunes) y "2 de 10" (Yoga, martes). Al confirmar una reserva el cupo baja y al cancelarla vuelve a subir.

JSON de la consulta de contacto. Al enviar el formulario se arma el objeto `{ nombre, email, mensaje, fecha }` (la fecha en formato ISO 8601), que es lo que se enviaría al servidor.

5. Uso de fetch y renderizado de la vista (puntos 4.3 y 4.4)

Todo el acceso a datos está en `js/modulos/api.js`. Cada archivo se pide con `fetch`, se comprueba que la respuesta sea correcta (`response.ok`) y que el contenido sea JSON con el formato esperado:

```
async function descargarJSON(ruta) {
  let respuesta;
  try {
    respuesta = await fetch(ruta, { headers: { Accept: 'application/json' } });
  } catch (causa) {
    throw new ErrorDeCarga(`No se pudo obtener ${ruta}.`, { cause: causa });
  }
  if (!respuesta.ok) {
    throw new ErrorDeCarga(`El servidor respondió ${respuesta.status} al pedir ${ruta}.`);
  }
  return await respuesta.json();   // si no es JSON válido, también lanza ErrorDeCarga
}
```

- **Carga en paralelo:** cada página pide lo que necesita con `Promise.all` (por ejemplo, clases y estado del usuario).
- **Caché:** una vez descargado, un archivo se reutiliza durante la visita. Los errores no se guardan en la caché, de modo que "Reintentar" vuelve a pedirlo.
- **Estados de la vista:** mientras se descarga se muestra "Cargando..."; si falla, un mensaje de error con botón *Reintentar*; si el filtro no encuentra resultados, un mensaje de "sin resultados".
- **Renderizado seguro:** los elementos se crean con `createElement` y el texto se inserta con `textContent`, nunca con `innerHTML`. Así, un dato del JSON o del usuario no puede inyectar HTML (se comprobó enviando una etiqueta img con onerror en el mensaje de contacto).
- **Filtros desde los datos:** las opciones de Disciplina y Día se generan a partir de `clases.json`, por lo que agregar una clase nueva no requiere tocar el HTML.

Página	Qué se dibuja desde JSON
Clases	Tarjetas del catálogo con una fila por día (día, cupos y botón de reserva), contador de resultados y panel "Tus Reservas Seleccionadas".
Pases	Filas de la tabla de planes con precio formateado (es-AR) y botón Comprar o "Plan activo".
Mi Cuenta	Nombre de usuario, plan activo (resuelto con pases.json) y lista de reservas (resueltas con clases.json).

6. Eventos (punto 4.1)

Evento	Elemento	Comportamiento
`submit`	Formulario de filtros	Se cancela el envío (`preventDefault`) y se filtra el catálogo sin recargar la página.
`change`	Selects de disciplina y día	Filtrado inmediato al elegir una opción.
`reset`	Botón Limpiar	Restablece los filtros y vuelve a mostrar todo el catálogo.
`click`	Reservar Cupo / Quitar	Agrega o quita la sesión de la selección. Un único listener en el contenedor atiende todos los botones (delegación de eventos).
`click`	Confirmar Reserva	Revalida cupos, registra las reservas y abre la pantalla de confirmación.
`click`	Comprar (Pases)	Pide confirmación con el precio y, si se acepta, cambia el plan activo.
`click`	Cancelar (Mi Cuenta)	Pide confirmación y libera el cupo.
`submit/blur/input`	Formulario de contacto	Validación y envío (sección 7).
`click/keydown`	Botón de menú	Abre y cierra el menú; Escape lo cierra y devuelve el foco al botón.
`change`	matchMedia (48em)	Al pasar a escritorio se restablece el estado del botón de menú.
`storage`	window	Si otra pestaña modifica las reservas, esta se actualiza sola.

```
refs.catalogo.addEventListener('click', (evento) => {
  const boton = evento.target.closest('button[data-accion]');
  if (!boton) return;
  alternarSesion(contexto, boton.dataset.accion, sesionDesdeClave(boton.dataset.sesion));
});
```

Tras cada cambio las vistas se vuelven a dibujar y el foco del teclado se devuelve al botón de la misma sesión, para que quien navega con teclado no lo pierda.

7. Validaciones del formulario de contacto (punto 4.5)

El formulario usa *novalidate* para reemplazar los globos del navegador por mensajes propios, ubicados bajo cada campo, con el mismo estilo que el resto del sitio. Las reglas están en `js/modulos/validacion.js` como funciones puras que devuelven el texto del error.

Campo	Reglas	Ejemplo de mensaje
Nombre	Obligatorio. De 2 a 60 caracteres. Solo letras (con tildes y ñ), espacios, apóstrofo, punto y guion.	El nombre solo puede contener letras y espacios.
Correo electrónico	Obligatorio. Formato texto@dominio.extensión, sin espacios.	Ingresa un correo válido, por ejemplo nombre@dominio.com.
Mensaje	Obligatorio. De 10 a 500 caracteres.	El mensaje debe tener al menos 10 caracteres (faltan 5).

- **Cuándo se valida:** al salir de un campo (blur); mientras el usuario corrige un campo con error (input), de modo que el mensaje desaparece apenas el valor es válido; y todos los campos al enviar.
- **Al enviar con errores:** se muestran todos los mensajes y el foco pasa al primer campo inválido.

- **Accesibilidad:** cada campo apunta a su mensaje con `aria-describedby`, se marca con `aria-invalid` y el borde rojo se agrega con la clase `formulario__control--invalido` (no depende solo del color: siempre hay texto).
- **Envío:** con datos válidos se construye el JSON, el botón pasa a "Enviando..." y queda deshabilitado (evita envíos dobles), se limpia el formulario y se muestra la pantalla "Mensaje enviado". No hay servidor, así que el envío se simula en `enviarConsulta()` (`api.js`); con un backend real se reemplazaría por un fetch con método POST.

8. Ciclo completo de reserva y compra

El catálogo exige un ciclo completo con envío y confirmación. En un gimnasio el "producto" es la reserva de una clase, por lo que la selección de clases cumple la función de carrito:

- **1. Filtrar** por disciplina y día.
- **2. Reservar Cupo** en el día deseado: la sesión pasa al panel "Tus Reservas Seleccionadas" y el botón cambia a "Quitar". La selección se conserva si se cambia de página.
- **3. Confirmar Reserva:** se revalidan los cupos y se muestra la pantalla "Reserva confirmada" con el detalle y el acceso a Mi Cuenta.
- **4. Mi Cuenta:** lista las reservas; *Cancelar* pide confirmación y devuelve el cupo.
- **Pases:** Comprar, confirmación con precio, pantalla "Compra confirmada" y actualización del plan activo en Mi Cuenta.

Regla	Resultado en la interfaz
Sesión sin cupos	Botón deshabilitado "Sin cupos" y etiqueta de cupos en rojo.
Sesión ya reservada	Botón deshabilitado "Reservada".
Pocos cupos (5 o menos)	Etiqueta de cupos resaltada en amarillo.
Dos clases el mismo día y horario	No se agrega y se avisa del conflicto.
Cupos agotados al confirmar	Se quita la sesión de la selección y se avisa.
Reserva de una clase que ya no existe en el JSON	Se descarta sin romper la página.
Plan actual en Pases	Botón deshabilitado "Plan activo".

Las pantallas de notificación usan el elemento *dialog* modal (bloquea el resto de la página, se cierra con Escape y devuelve el foco al elemento que lo abrió). Los avisos breves ("Agregada a tu selección...") se anuncian a lectores de pantalla mediante una región `aria-live`.

9. Modularización del código (punto 4.7)

Todas las páginas cargan un único script, `<script type="module" src="js/main.js">`. Cada *body* declara su página con `data-pagina` y `main.js` inicia lo común (menú y avisos) y carga con `import()` dinámico solo el módulo de esa página: Contacto no descarga el código de Clases, y Inicio no descarga ninguno.

Módulo	Responsabilidad
`modulos/api.js`	fetch de los JSON, validación de formato, caché y errores de carga; envío simulado de consultas.
`modulos/estado.js`	Plan, reservas y selección; persistencia en localStorage; suscripción a cambios.
`modulos/reglas.js`	Cupos, estado de cada sesión, conflictos, filtros (funciones puras).

Módulo	Responsabilidad
`modulos/validacion.js`	Reglas del formulario de contacto (funciones puras).
`modulos/formato.js`	Precios, nombres de días y textos de reservas (funciones puras).
`modulos/dom.js`	Creación segura de elementos (sin innerHTML).
`modulos/almacenamiento.js`	Lectura y escritura de localStorage tolerante a fallos.
`componentes/dialogo.js`	Pantallas de confirmación y notificación (dialog).
`componentes/avisos.js`	Avisos breves (toasts).
`componentes/menu.js`	Menú desplegable en móvil.
`componentes/mensaje-estado.js`	Mensajes de carga, error y sin resultados.
`paginas/clases.js`	Catálogo, filtros, selección y confirmación de reservas.
`paginas/pases.js`	Tabla de planes y compra.
`paginas/cuenta.js`	Datos del usuario y cancelación de reservas.
`paginas/contacto.js`	Validación y envío del formulario.
`main.js`	Punto de entrada: elige y carga el módulo de cada página.

10. Capturas

Las capturas se tomaron con el sitio funcionando sobre un servidor local, en 1440 px (escritorio) y 375 px (móvil).

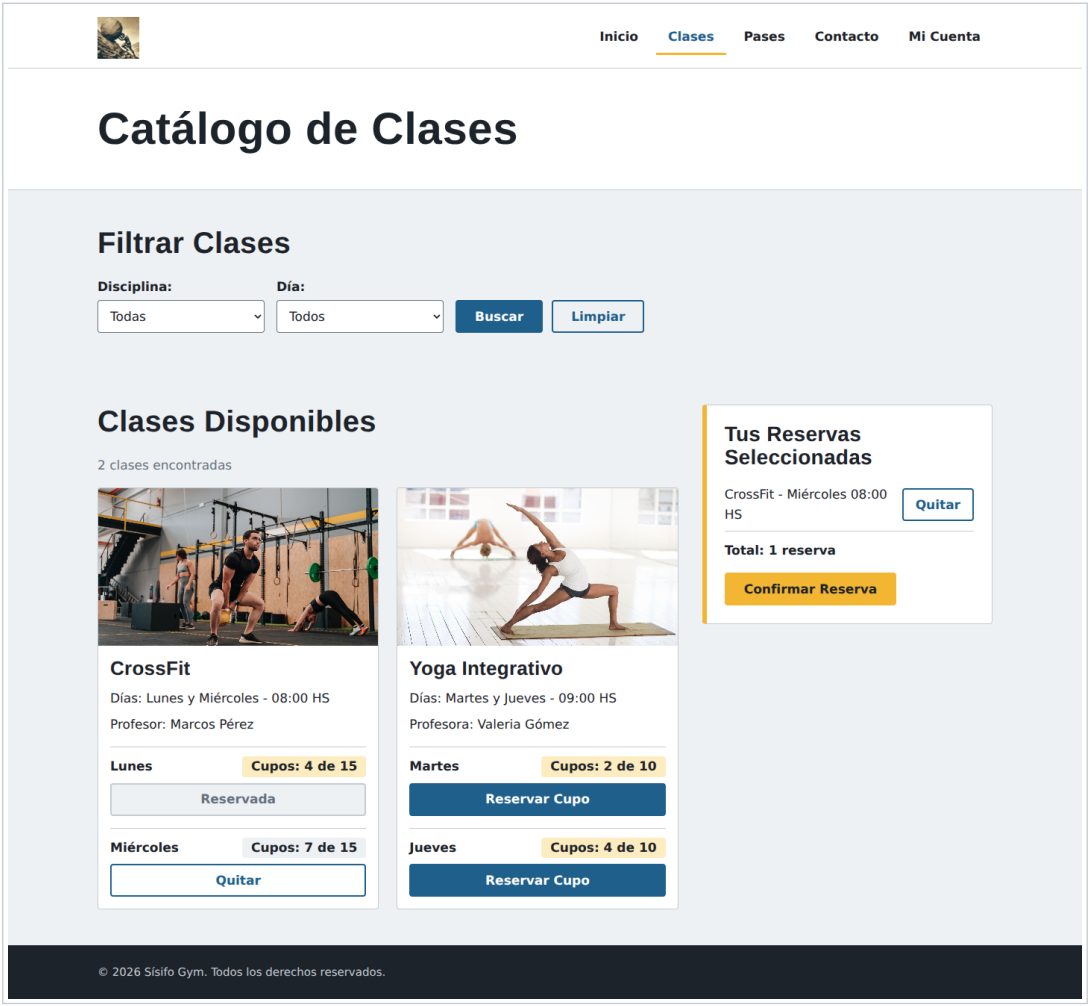


Figura 1. Clases (escritorio): catálogo desde JSON, filtros y panel con una clase seleccionada.

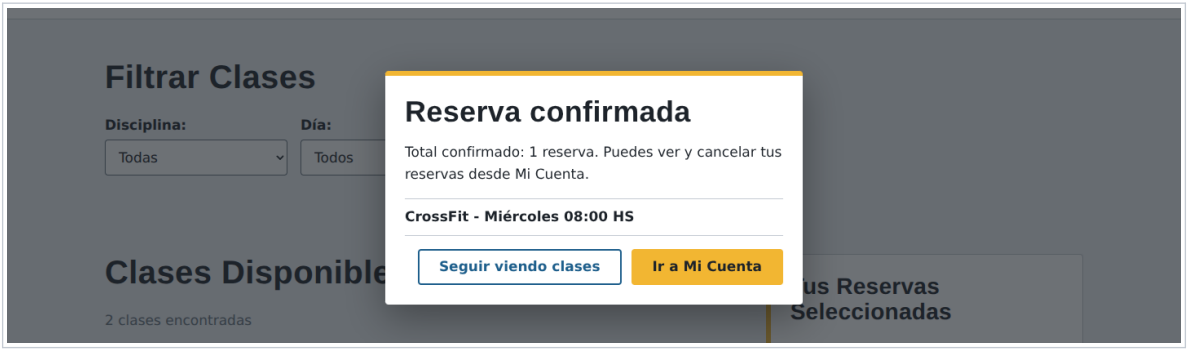


Figura 2. Pantalla "Reserva confirmada" (elemento dialog modal).

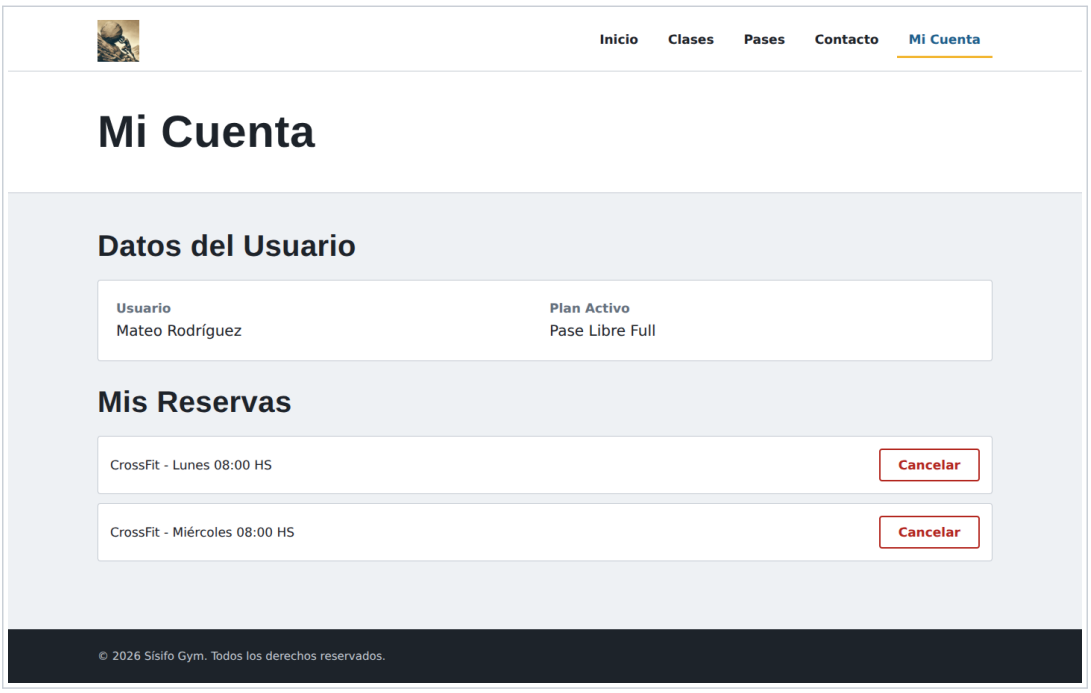


Figura 3. Mi Cuenta (escritorio): plan activo y reservas con botón Cancelar.

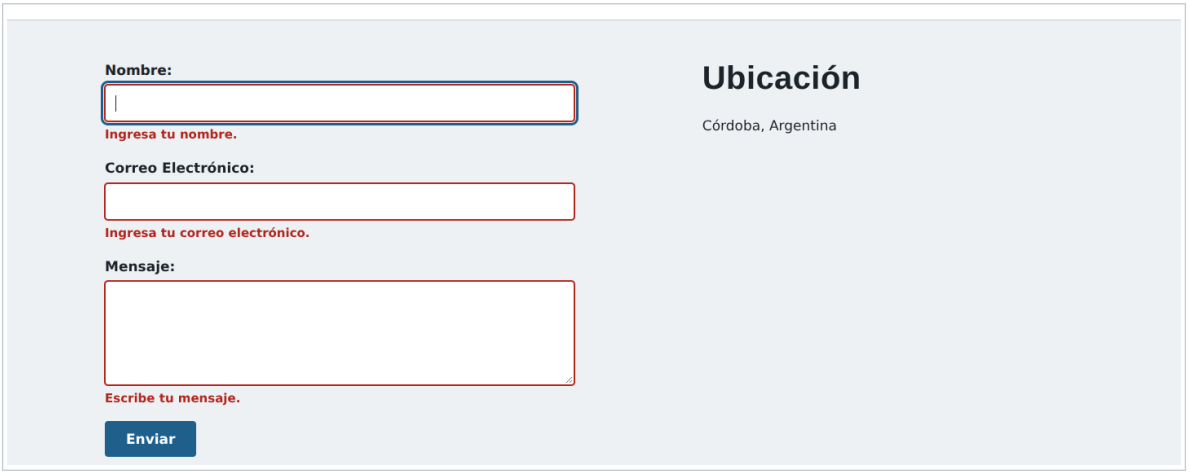


Figura 4. Contacto: validación al enviar el formulario vacío; el foco queda en el primer campo inválido.

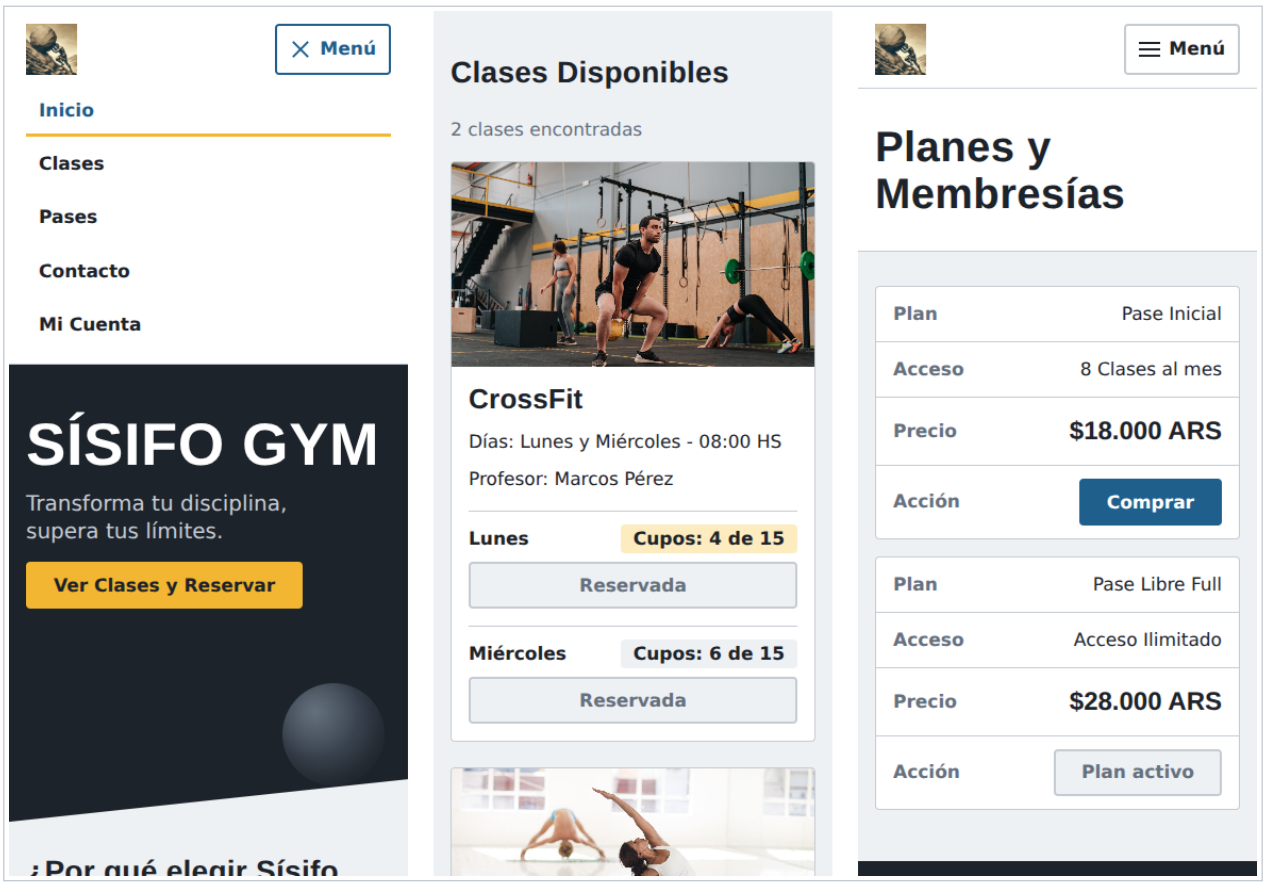


Figura 5. Móvil (375 px): menú desplegable abierto, catálogo con sesiones por día y tabla de pases como tarjetas.

11. Cambios respecto a la entrega anterior (SP3)

El diseño visual, los textos y la estructura de las páginas se mantienen. Los cambios necesarios para integrar JavaScript fueron:

HTML

- **Todas las páginas:** botón de menú (`cabecera__menu-boton`) con `aria-expanded` y `aria-controls`; id en la navegación; `data-pagina` en el body; script `type="module"`; y un script de una línea en el head que agrega la clase `js` a la etiqueta html (mejora progresiva: sin JavaScript el menú sigue visible).
- **Clases:** los datos de las tarjetas y las opciones de los filtros ya no están escritos en el HTML. Los filtros ahora incluyen Miércoles y Jueves (antes solo Lunes y Martes, aunque las clases se dictan también esos días). Se agregó el botón *Limpiar* y un contador de resultados.
- **Clases (reservas):** cada tarjeta muestra una fila por día con sus cupos y su botón, porque una clase se dicta en dos días y hay que elegir cuál reservar (antes había un solo botón por clase y los cupos eran por clase).
- **Clases (panel):** "Confirmar Reserva" pasó de ser un enlace a Mi Cuenta a ser un botón que confirma y muestra la pantalla de confirmación; permanece deshabilitado si no hay selección.
- **Pases y Mi Cuenta:** el cuerpo de la tabla, el nombre de usuario, el plan y las reservas se llenan con JavaScript. La tabla permanece oculta hasta que los datos cargan.
- **Contacto:** atributo `novalidate`, un párrafo de error por campo, `aria-describedby` y `maxlength`.
- **Clases, Pases y Mi Cuenta:** se agregó `noscript` para avisar que estas páginas necesitan JavaScript.

CSS

- Nuevos bloques con la misma nomenclatura BEM: `cabecera__menu-boton`, `tarjeta-clase__sesion(es)`, `mensaje-estado`, `aviso(s)`, `dialogo`, `formulario__error`.

- Nuevos modificadores: `boton--compacto`, `tarjeta-clase__cupos--agotado`, `formulario__control--invalido`; estilo para botones deshabilitados y variable `--color-exito`.
- Regla global `[hidden] { display: none !important; }` para que el atributo hidden funcione aunque un componente defina su propio display.
- Menú desplegable con enfoque mobile first: en móvil (con JavaScript) el menú ocupa una fila propia y se despliega; desde 48em vuelve a ser una fila siempre visible.

Datos y decisiones

- Los textos de las clases, los planes y el usuario no cambian: ahora viven en los JSON. Se verificó que los cupos iniciales coincidan con los del diseño anterior.
- El estado (reservas, selección y plan) se guarda en localStorage para que persista entre páginas, ya que el sitio no tiene servidor.

12. Pruebas realizadas

Se probó el sitio en Chromium a 375 px y 1440 px. Se automatizaron 82 verificaciones del recorrido completo (carga desde JSON, filtros, selección, confirmación, cancelación, compra de pase, validación y envío del formulario, menú móvil, ausencia de desplazamiento horizontal, falla de carga con reintento y ausencia de errores de JavaScript) y 12 pruebas unitarias sobre las reglas de cupos, conflictos, filtros, validación y formato. No se probó en otros navegadores.

13. Limitaciones y pendientes

- No hay servidor: el estado vive en el navegador de cada persona y el envío del formulario es una simulación. Con un backend, `api.js` sería el único archivo a adaptar.
- El límite de 8 clases al mes del Pase Inicial no se controla al reservar.
- Los conflictos se detectan solo cuando dos clases coinciden en día y horario exactos (los datos no incluyen duración).
- El catálogo de ejemplo tiene dos clases; la estructura admite más sin cambiar el código.
- Queda como etapa opcional el despliegue de la aplicación (punto 5 de las consignas).